Locating Visual Information in Hybrid Linear-Attention Vision-Language Models: Training-Free Interventions on the KV Cache, Recurrent State and Convolution Cache

Minseok Shin
Horcrux Technologies, Seoul, Republic of Korea
horcruxtech.ai@gmail.com stevenshin16@korea.ac.kr

Abstract

Hybrid vision-language models that interleave linear-attention layers with softmax attention read an image into two memories at once: the key-value (KV) rows of the visual tokens in the attention layers, and a fixed-size recurrent state in the linear-attention layers that has absorbed every visual token. Visual-token pruning and multimodal KV-cache compression assume that the image lives in the first of these. We ask where it actually lives. On Qwen3.5-0.8B (a Gated DeltaNet hybrid with early fusion) we alter one memory exactly once, right after the last image token, and let the question and the answer read the altered memory. A dataset-level ratio of task accuracy, anchored at a no-image run and an unaltered run, measures how much of the image's contribution each memory retains, with paired bootstrap intervals, on DocVQA, ChartQA, TextVQA and POPE; the same ratio of teacher-forced answer losses is reported alongside. Removing the visual KV rows leaves at most 7% of the image's gain on the three reading tasks and 53% on POPE; removing the recurrent state leaves 66–97% while the convolution cache still passes on the last visual tokens, and 33–69% once it is cleared too, so the state matters for reading but cannot replace the KV rows. Both memories are robust to precision after reading: 4-bit KV with a 6-bit state keeps 96–101% of the gain while cutting the stored cache by 4.6–5.0×, and error feedback adds nothing measurable. At equal cost per visual token, 4-bit KV for every token keeps 98–100%, while keeping a quarter of the tokens at 16 bits keeps 31–48% on reading tasks. Clearing all three carriers returns the model to its no-image accuracy. Finally, the state shapes the form of the answer: zeroing it moves POPE answers from lower to upper case, which flips the sign of a loss-based retained ratio while accuracy barely changes. A check on Qwen3.5-4B (100 samples per task) agrees: without the visual KV rows at most 6% of the gain is left on reading tasks, and without the state none of it is lost.

1 Introduction

A transformer vision-language model (VLM) keeps everything it has read about an image in one place: the keys and values of the visual tokens, one row per token and layer. Visual-token pruning [1–5] and multimodal KV-cache compression [6, 7] are built on that picture: removing or compressing a visual token's row removes or compresses what the model knows about that part of the image.

Hybrid models break the picture. Qwen3.5 [8] repeats three Gated DeltaNet (GDN) layers [9] and one gated softmax-attention layer, and it is natively multimodal: visual features enter at the input embedding and pass through the same stack as text (early fusion; Section 3.1). When the last image token has been processed, the image has been written into two memories. The attention layers hold the KV rows of the visual tokens, as in a transformer. The GDN layers hold a fixed-size recurrent state that has been updated by every visual token and is not indexed by token at all. In our setting this state is larger than the visual KV rows of any image we feed (Section 3.3).

Which of the two memories does the model consult when it answers a question about the image? The answer decides what token pruning and KV compression can safely do to a hybrid VLM, and whether the recurrent state, which a serving engine stores at full precision regardless of the KV format

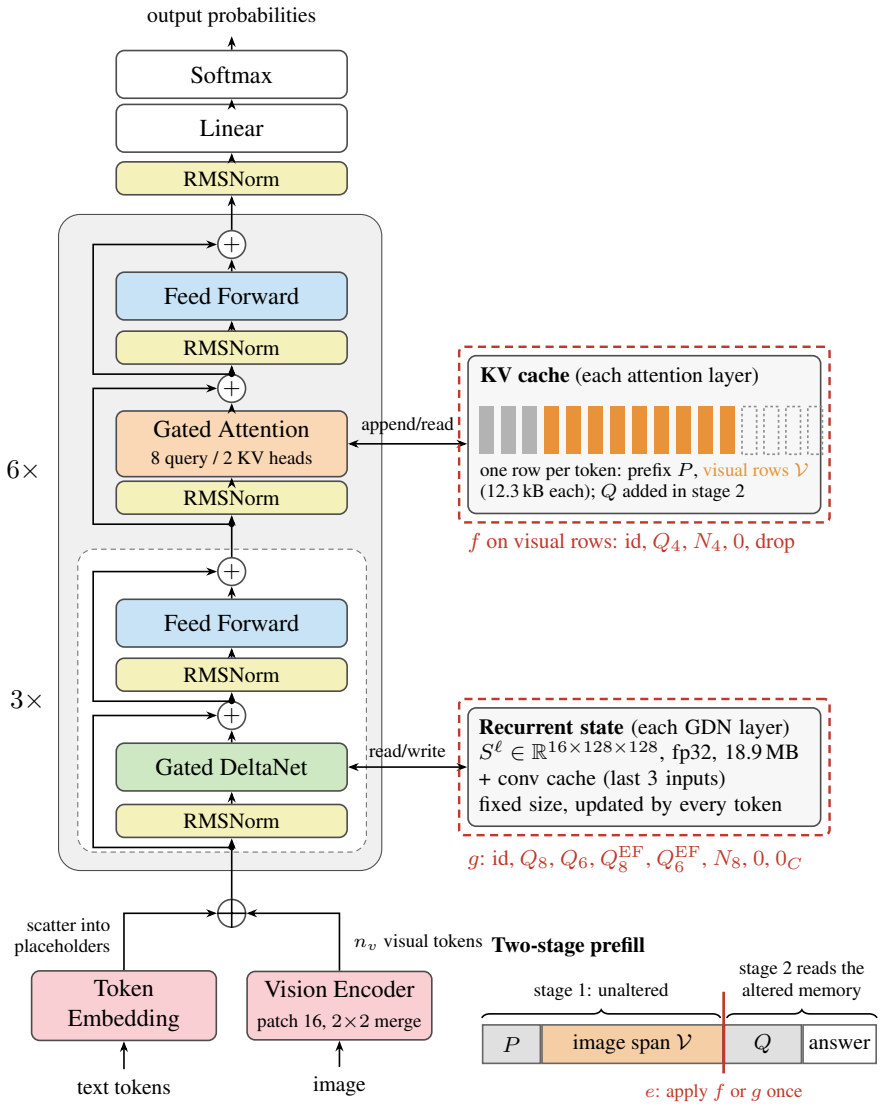


Figure 1: Qwen3.5 as a hybrid VLM and the two memories it keeps. Left: the decoder stack (pre-norm layers, six groups of three Gated DeltaNet layers and one gated attention layer for 0.8B; eight for 4B and 9B); visual tokens from the vision encoder replace placeholder tokens at the input embedding (early fusion). Right: the recurrent state of each GDN layer (fixed size, updated by every token) and the KV cache of each attention layer (one row per token, visual rows in orange). The prompt is read in two stages; right after the last image token (e) one memory is altered once, by f on the visual KV rows or by g on the state (red), and the question and the answer then read the altered memory.

[10], is a place where image information can be lost. It is not obvious a priori. The state is written by every visual token but has a finite capacity and forgets through a learned decay; the KV rows keep every token exactly but sit in only one layer in four.

We answer the question with training-free interventions (Figure 1). The prompt is read in two stages: the prefix and the image first, then the question (Section 4.1). Between the stages we alter exactly one memory once: we zero it, quantize it, or replace it by matched noise, or we remove visual tokens before they are read. The question and the answer then read the altered memory. We measure the effect with the task accuracy of the greedy answer, normalised between a run without the image and an unaltered run (Section 4.3), so that 1 means the altered memory still supplies everything the image contributed and 0 means nothing is left; the same ratio of the teacher-forced loss of the reference answer is reported alongside.

Two text-only studies of the same model family frame the question. Split-prefill experiments on Qwen3.5 and Falcon-H1 keep only the KV cache or only the recurrent state of a prefilled context and find that exact retrieval survives only through attention, while the output language and a persona survive through the recurrent state [11]. Quantizing both

memories at every write found the recurrent state to be the fragile one under long generation [12]. Neither involves images. An image differs from a text context in two ways that matter here: it is long (up to 1,280 tokens in our setting) and dense, and the questions asked about it range from reading a single word (DocVQA, TextVQA) to judging whether an object is present at all (POPE). The retrieval-versus-gist split of [11] predicts that reading tasks depend on the KV rows and presence judgements less so; our tasks test that prediction for visual content. For hybrid Mamba–Transformer video models, token reduction has been found to behave like compression rather than dropping because the state keeps information from discarded tokens [13]; we measure that state contribution directly, for still images and Gated DeltaNet.

Contributions.

- A one-shot intervention protocol for hybrid VLMs that separates the two memories at the end of the image span, with a dataset-level retained ratio and paired bootstrap intervals (Sections 4.1–4.3).
- Reading tasks need the visual KV rows: without them the recurrent state supplies at most 7% of the image's gain in accuracy, against 53% for object presence (Section 6.1).
- Three carriers, not two: the convolution cache of the GDN layers raises what is left after zeroing the state from 33–69% to 66–97%, and removing KV rows, state and convolution cache together returns the model to its no-image accuracy (Section 6.1).
- After reading, 4-bit KV and a 6-bit state are lossless within the bootstrap intervals, without error feedback (Section 6.2); at equal KV cost, quantizing every visual token beats keeping fewer tokens by 50–70 points of retained gain on reading tasks (Section 6.3).
- A loss-based retained ratio can invert when an intervention changes the answer's form rather than its content; we show this for letter case on POPE and report accuracy as the primary measure (Section 6.5).
- At 4B the split is sharper: removing the visual KV rows leaves 5–6% on reading tasks, and zeroing the state (convolution cache kept) leaves the full gain on every task (Section 6.4).

2 Related Work

Visual-token reduction. FastV drops visual tokens after an early layer on the evidence of low attention to them [1]; SparseVLM ranks visual tokens with text-guided attention [2]; LLaVA-PruMerge and VisionZip select and merge tokens at the vision encoder output [3, 4]; PyramidDrop removes tokens progressively with depth [5]; token merging goes back to ToMe [14]. All of these are designed and evaluated on transformer language backbones, where the visual-token rows are the only memory of the image. For Mamba-based multimodal models, DTP scores visual tokens from the selective-scan parameters instead of attention [15]; for hybrid video models, token reduction is reported to act as compression because the state keeps what dropped tokens contributed [13].

Multimodal KV-cache compression. LOOK-M and VL-Cache compress the KV cache of multimodal inputs with modality-aware eviction and budget allocation [6, 7]. KV quantization for text, e.g. KIVI [16] and KVQuant [17], applies unchanged to the visual rows we quantize. CalibQuant quantizes the visual KV cache of transformer VLMs down to 1 bit and argues that, unlike token dropping, quantization keeps every visual token [18]; we test that trade-off at matched cost in a hybrid model.

Where visual information flows. Attention-knockout and patching studies of transformer VLMs trace how image information reaches the answer. In InternVL2-76B and LLaVA-1.5-7B, the query tokens come to store global image information, cross-modal flow is concentrated in about a quarter of the layers in the middle of the network, and fine-grained details are read from the image tokens directly [19]. In LLaVA models, general visual features move into the question tokens in lower layers and question-relevant object information in middle layers before reaching the last position [20]; removing object-specific visual tokens lowers object identification by more than 70% [21]. Causal patching separates a direct pathway, read from the image tokens, from a text-mediated pathway through the query tokens, chosen per task, with the second acting as a fallback when the first is ablated [22]. In a hybrid model the text-mediated route has a second carrier: the recurrent state delivers image information to the question tokens without any attention to the image rows. Zeroing the visual KV rows removes the direct pathway in the attention layers and leaves this route intact; zeroing the state does the converse.

The two memories of hybrid models. Afendulev et al. separate the KV cache and the recurrent state of text contexts by keeping one and dropping the other (the convolution cache is dropped with the state) and by swapping states between contexts [11]. We differ in the input (an image span inside the prompt, followed by the question), in the interventions (graded: quantization, error feedback, matched noise and token deletion besides removal) and in the measure (a loss ratio anchored at no-image and full runs). Hybrid video models have used the state as evidence for which visual tokens to keep: as a carrier of discarded tokens [13] and, through its change over a chunk, as a query-free eviction signal for the KV cache [23].

Linear and hybrid VLMs; recurrent-state compression. Mamba-based VLMs replace the language backbone by a state-space model [24, 25]; there the state is the only memory of the image. InfiniteVL interleaves a small fraction of full-attention layers with Gated DeltaNet in a VLM built by distillation [26]; Qwen3.5 is natively pretrained as a hybrid on text and images [8]. Post-training quantization of the recurrent state of GDN and related layers has been studied for text [27–29], and jointly with the KV cache in [12]. None of these isolates the image.

Table 1: Cache geometry, from the model configurations. Sizes by (4); KV in bf16, state in fp32.

Model	L	L_a	L_g	H_{kv}	d	H	$d_k{=}d_v$	$\lvert S\rvert$ (MB)
Qwen3.5-0.8B	24	6	18	2	256	16	128	18.9
Qwen3.5-4B	32	8	24	4	256	32	128	50.3
Qwen3.5-9B	32	8	24	4	256	32	128	50.3

GDN key heads: 16 in all three; the state is per value head. Query heads: 8 (0.8B), 16 (4B, 9B). Vision encoder: patch 16, merge 2; depth 12 (0.8B), 24 (4B), 27 (9B). Only 0.8B is run in this version.

3 Background

3.1 Qwen3.5 as a hybrid VLM

Qwen3.5 has L decoder layers in repeating groups of three GDN layers and one gated softmax-attention layer, so $L_a = L/4$ attention and $L_g = 3L/4$ GDN layers. For the 0.8B model we study $L = 24$, $L_a = 6$, $L_g = 18$; the 4B and 9B models have $L = 32$ (Table 1).

Early fusion. A vision encoder with patch size $p = 16$ and $m \times m$ spatial merging ($m = 2$) turns an image resized to $H \times W$ pixels (both multiples of $pm = 32$) into

$$n_v = \frac{H}{p} \cdot \frac{W}{p} \cdot \frac{1}{m^2} = \frac{HW}{(pm)^2} \tag{1}$$

visual tokens, which replace n_v placeholder tokens in the input embedding. In the transformers implementation we use, this scatter at the input embedding is the only place where visual features enter the language model; no visual features are injected at intermediate layers (Appendix A, premise 1). Visual tokens get three-dimensional rotary positions (temporal, height, width; M-RoPE [30]) on the first quarter of each attention head's dimensions. We cap the image at $HW \leq 1{,}310{,}720$ pixels, so $n_v \leq 1{,}280$.

3.2 Gated DeltaNet

Each GDN head keeps a matrix $S_t \in \mathbb{R}^{d_k \times d_v}$. With L2-normalised key $k_t \in \mathbb{R}^{d_k}$, query $q_t \in \mathbb{R}^{d_k}$ (further scaled by $d_k^{-1/2}$), value $v_t \in \mathbb{R}^{d_v}$, decay $\alpha_t \in (0,1)$ and write strength $\beta_t \in (0,1)$, computed per head from the input at t as $\alpha_t = \exp\bigl(-e^{A}\,\mathrm{softplus}(a_t{+}b)\bigr)$ and $\beta_t = \mathrm{sigmoid}(\beta'_t)$ with a_t, β'_t linear in the input and A, b learned, the state is first decayed and then corrected along k_t:

$$\begin{aligned} S_t &= \alpha_t S_{t-1} + \beta_t\, k_t \bigl(v_t - \alpha_t S_{t-1}^{\top} k_t\bigr)^{\top} \\ &= \alpha_t \bigl(I - \beta_t k_t k_t^{\top}\bigr) S_{t-1} + \beta_t\, k_t v_t^{\top}, \qquad (2) \\ o_t &= S_t^{\top} q_t. \qquad (3) \end{aligned}$$

This is the order of operations of the reference recurrence in the implementation (decay, read $(\alpha_t S_{t-1})^{\top} k_t$, write). Prefill computes the same recurrence in chunks, but the state is written to the cache once per forward call, at its end (premise 5). Before the recurrence, q, k, v pass through a causal depthwise convolution of kernel size $\omega = 4$ whose cache holds the last inputs; we return to it in Section 4.5.

3.3 Two memories and their sizes

With H_{kv} KV heads of dimension d per attention layer and H GDN value heads, a sequence of T tokens leaves

$$|\mathrm{KV}|(T) = 2L_a H_{kv} d\, T, \qquad |S| = L_g H d_k d_v \tag{4}$$

elements in the two memories (the factor 2 counts keys and values). The implementation stores the KV cache in the model dtype (bf16, 16 bits) and the recurrent state in fp32 (32 bits; premise 3). For 0.8B, $|\mathrm{KV}|(T) = 6{,}144\,T$ and $|S| = 18{\cdot}16{\cdot}128{\cdot}128 = 4{,}718{,}592$, i.e. 12.3 kB of KV per token and an 18.9 MB state. The visual rows of an image therefore occupy $0.0123\,n_v$ MB, at most 15.7 MB at $n_v = 1{,}280$, and the two are equal only at

$$n_v^{\star} = \frac{32\,|S|}{16 \cdot 2L_a H_{kv} d} = 1{,}536 \tag{5}$$

visual tokens. The 4B and 9B models have both memories $8/3$ times larger (32.8 kB per token, 50.3 MB) and the same crossover. Every image in this study leaves a state larger than its own KV rows. (The crossover of 768 tokens in [12] counts the state in bf16; the implementation used here keeps it in fp32.)

4 Method

4.1 Two-stage prefill and the intervention point

A prompt is the token sequence $x = (P, \mathcal{V}, Q)$: a prefix P (chat template up to and including the vision-start token), the image span $\mathcal{V} = \{s, \ldots, e-1\}$ of $n_v = e - s$ placeholder positions carrying the visual features, and the question Q (vision-end token, question, instruction and the assistant header). Let $\mathcal{M}_e$ be the state of all caches after the first e tokens:

$$\mathcal{M}_e = \bigl(\{K^{\ell}_{1:e}, V^{\ell}_{1:e}\}_{\ell \in \mathcal{A}},\ \{S^{\ell}_e,\, C^{\ell}_e\}_{\ell \in \mathcal{G}}\bigr), \tag{6}$$

with $\mathcal{A}$, $\mathcal{G}$ the attention and GDN layers and C_e^ℓ the convolution cache. Stage 1 runs $(P, \mathcal{V})$ with unaltered computation. An intervention ϕ maps $\mathcal{M}_e$ to $\phi(\mathcal{M}_e)$. Stage 2 runs Q on $\phi(\mathcal{M}_e)$ without further alteration; the reference answer is then scored by teacher forcing on a copy of the resulting cache, and a greedy answer of at most 16 tokens is decoded on the original. Because nothing is altered after the image, any effect of ϕ is an effect on what the image left behind. All conditions, including the unaltered one, use the same two-stage path. On the first 50 samples of each task, the unaltered two-stage run and the stock single-pass forward give mean gold losses that differ by at most 0.007 nats per token (0.21% of the blind–full gap; per-sample differences up to 0.32 from bf16 rounding of the split), so the split itself does not move the anchors.

4.2 Interventions

KV side. For every attention layer, the rows of the visual positions are replaced, $K_\mathcal{V}^\ell \leftarrow f(K_\mathcal{V}^\ell)$ and $V_\mathcal{V}^\ell \leftarrow f(V_\mathcal{V}^\ell)$, with $f \in \{\mathrm{id},\, Q_4,\, N_4,\, 0\}$, or the visual rows are removed from the cache altogether (*drop*). Rows of P are not touched, and rows of Q are written later by stage 2. After a drop, the causal-mask positions of later tokens follow the shorter cache while their rotary positions are unchanged, so the result equals keeping the rows and masking their attention columns (verified to 10^{-4} on a test model). Keys are cached after the rotary embedding, so f acts on rotated keys.

State side. For every GDN layer, $S_e^\ell \leftarrow g(S_e^\ell)$ with $g \in \{\mathrm{id},\, Q_8,\, Q_6,\, Q_8^{\mathrm{EF}},\, Q_6^{\mathrm{EF}},\, N_8,\, 0\}$; these leave the convolution cache C_e^ℓ in place. A last level, 0_C, zeroes both S_e^ℓ and C_e^ℓ, so that nothing of the image survives in the GDN layers.

Quantizer. Q_b is asymmetric uniform round-to-nearest with one scale and zero point per group of $G = 128$ consecutive elements along the last dimension (the head dimension $d = 256$ for keys and values, two groups per head and token; d_v for the state, one group per row $S[h, i, :]$). For a group $x \in \mathbb{R}^G$,

$$\begin{aligned} \sigma &= \frac{\max x - \min x}{2^b - 1}, \qquad z = \lfloor -\min x/\sigma \rceil, \\ Q_b(x) &= \sigma\Big(\mathrm{clamp}\big(\lfloor x/\sigma \rceil + z,\, 0,\, 2^b - 1\big) - z\Big), \end{aligned} \tag{7}$$

where $\lfloor \cdot \rceil$ rounds to the nearest integer and σ is floored at 10^{-8}. Quantization is simulated in fp32 and cast back to the cache dtype. Because the grid spans the group's range, every element is within half a step of its stored value, $|Q_b(x)_i - x_i| \le \sigma/2$, i.e. at most $1/(2(2^b - 1))$ of the group range: 3.3% at 4 bits, 0.79% at 6 bits and 0.20% at 8 bits.

Matched noise. N_b adds Gaussian noise with the energy of b-bit quantization error but none of its structure. For a cache tensor X (one layer's visual keys, values, or state),

$$\begin{aligned} N_b(X) &= X + \rho_b(X)\, Z, \\ \rho_b(X) &= \sqrt{\tfrac{1}{|X|} \textstyle\sum \big(Q_b(X) - X\big)^2}, \end{aligned} \tag{8}$$

with Z standard normal entries of the shape of X. The generator is re-seeded (seed 0) for every tensor, so tensors of the same shape receive the same Z, across layers and samples, scaled by their own ρ_b. The RMS ρ_b is taken over the whole tensor, so noise is spread uniformly where quantization error is concentrated in high-range groups.

Error feedback during the image. Quantizing the state once after the image (Q_b above) commits one rounding error. With error feedback the image is read in chunks of at most 256 visual tokens, with boundaries $s = e_0 < e_1 < \cdots < e_C = e$, $C = \lceil n_v/256 \rceil \le 5$. Let $\hat{S}_{e_u}$ be the state after running chunk u from the stored state $\tilde{S}_{e_{u-1}}$, with $\tilde{S}_{e_0} = S_s$ the unaltered state after the prefix. Then

$$\tilde{S}_{e_u} = Q_b\big(\hat{S}_{e_u} + r_{u-1}\big), \quad r_u = \hat{S}_{e_u} + r_{u-1} - \tilde{S}_{e_u}, \tag{9}$$

with $r_0 = 0$; the final residual r_C is discarded and $\tilde{S}_e$ is what the question reads. Each chunk is a separate forward call, because the state is written once per call (premise 5). For $n_v \le 256$, $C = 1$ and $Q_b^{\mathrm{EF}} = Q_b$. Error feedback here therefore acts across at most five stores, unlike decoding in [12], where it acts across every generated token.

Zeroing and removal. $f = 0$ sets visual keys and values to zero and $f =$ drop removes them; $g = 0$ sets every GDN state to zero and $g = 0_C$ also its convolution cache. Section 4.5 states what each removes and what it leaves; the pairs (0, drop) and $(0, 0_C)$ separate the two side effects from the removal itself.

Visual-token deletion. A keep set $I \subset \{0, \ldots, n_v - 1\}$ of size $n_v' = \max(1, \mathrm{round}(\kappa n_v))$, $\kappa \in \{0.5, 0.25\}$, is chosen uniformly, $I = \{\lfloor j\, n_v/n_v' \rfloor : j = 0, \ldots, n_v' - 1\}$, or at random without replacement (seed 0). The other visual tokens are removed from the input before the language model, so neither memory ever sees them; the kept tokens keep their original M-RoPE positions, and the vision encoder still sees the full image. Deletion shrinks the visual KV rows to n_v' and leaves $|S|$ unchanged.

Every-write quantization. For comparison with [12] we also quantize every KV append and every state write of the whole run with the released hooks of that work (scope *whole*); the image and the text are quantized alike. The state is then stored at the end of stage 1, at the end of stage 2, after the teacher-forced pass over the reference answer, and after every decoded token. With error feedback the residual is reset at the first store of each layer (end of stage 1) and carried through the later stores. The residual buffer is shared between

the teacher-forced copy of the cache and the decoding cache, so the first decoding step continues from the residual of the teacher-forced pass; this affects the greedy answers of the two every-write cells with error feedback, not their losses. We report these two cells as run.

4.3 Retained ratio

We report two retained ratios with the same anchors. Two anchors use the same prompt: *full* (image, no alteration) and *blind* (no image: the image span and its start and end tokens are removed).

Accuracy retained (primary). With $\bar{s}(c)$ the mean task score (defined below) of the greedy answers to a task's n samples under condition c,

$$\mathcal{R}(c) = \frac{\bar{s}(c) - \bar{s}(\text{blind})}{\bar{s}(\text{full}) - \bar{s}(\text{blind})}. \tag{10}$$

$\mathcal{R} = 1$: the altered memory supplies the full gain in task score that the image provides; $\mathcal{R} = 0$: none of it; $\mathcal{R} < 0$: worse than no image. It is defined when $\bar{s}(\text{full}) > \bar{s}(\text{blind})$, which we check per task before reporting.

Loss retained (secondary). Let the reference answer have tokens $a_{1:m}$. For sample i under condition c,

$$\text{NLL}_i(c) = -\frac{1}{m}\sum_{j=1}^{m} \log p_c\big(a_j \mid x,\, a_{<j}\big), \tag{11}$$

where p_c reads the memory produced by condition c. With $\bar{N}(c) = \frac{1}{n}\sum_i \text{NLL}_i(c)$ over a task's n samples,

$$\mathcal{R}_{\text{NLL}}(c) = \frac{\bar{N}(\text{blind}) - \bar{N}(c)}{\bar{N}(\text{blind}) - \bar{N}(\text{full})}, \tag{12}$$

read as $\mathcal{R}$ with the reduction in answer loss in place of the gain in score. Both ratios are ratios of dataset means, not means of per-sample ratios, whose denominators can be near zero (for scores in $\{0, 1\}$ they are undefined for most samples). Writing B_i, F_i, C_i for the three per-sample losses,

$$\begin{aligned} \mathcal{R}_{\text{NLL}}(c) &= \frac{\sum_i (B_i - C_i)}{\sum_i (B_i - F_i)} = \sum_i \omega_i \frac{B_i - C_i}{B_i - F_i}, \\ \omega_i &= \frac{B_i - F_i}{\sum_{i'} (B_{i'} - F_{i'})}, \end{aligned} \tag{13}$$

where the second equality requires $B_i \neq F_i$ for every sample. $\mathcal{R}_{\text{NLL}}$ thus weights each sample by how much the image helps it: samples the image barely helps contribute little, and samples it hurts ($B_i < F_i$) enter with negative weight. Every sample's loss is a per-token mean, so long answers do not dominate. It is defined when $\bar{N}(\text{blind}) > \bar{N}(\text{full})$, which we check per task before reporting. For both ratios the 95% interval is a paired percentile bootstrap [31]: 2,000 resamples of sample indices with replacement (seed 0), the same indices for the three means; resamples with a zero denominator are dropped.

Reference answers and task scores. The reference for the loss is the first answer (DocVQA), the most frequent of the ten annotator answers (TextVQA), or the single answer (ChartQA, POPE). Greedy answers are scored with ANLS at $\tau = 0.5$ [32] (DocVQA), relaxed accuracy with 5% relative tolerance [33, 34] (ChartQA), VQA accuracy $\frac{1}{10}\sum_{i=1}^{10} \min\big(1, \frac{1}{3}\#\{j \neq i : a^{(j)} = \hat{a}\}\big)$ [35] (TextVQA), and yes/no accuracy and F1 with *yes* as the positive class (POPE), all after one shared normalisation (lower case; punctuation except decimal points and percent signs removed; articles removed).

Why accuracy is primary. The loss is taken on the reference string as released, including its letter case, whereas the scores are computed after lower-casing. The two disagree where the model's own casing differs from the reference: in the unaltered condition, the greedy answer differs from the reference only in letter case for 37% of POPE samples (references are lower-case *yes*/*no*, the model writes *Yes*/*No*) and 20% of DocVQA samples, against under 1% for ChartQA and TextVQA. On those samples NLL_i measures the cost of an unusual casing as well as of a wrong answer, so a condition that only shifts casing moves $\mathcal{R}_{\text{NLL}}$ and not $\mathcal{R}$; $\mathcal{R}_{\text{NLL}}$ can then exceed 1 or change sign while the answers stay correct. We therefore draw conclusions from $\mathcal{R}$, report $\mathcal{R}_{\text{NLL}}$ alongside, and mark the POPE and DocVQA columns of $\mathcal{R}_{\text{NLL}}$.

4.4 Memory accounting

For the equal-memory comparison we charge a condition

$$\text{MB}(c) = \frac{b_{kv}\,|\text{KV}|(n'_v + n_t) + b_s\,|S|}{8 \cdot 10^6}, \tag{14}$$

with n'_v the visual tokens kept, n_t the text tokens, $b_{kv} \in \{16, 4, 0\}$ (full, 4-bit, zero) and $b_s \in \{32, 8, 6, 0\}$; noise is charged at its matched bit-width. This is deployment accounting: all KV rows are charged at b_{kv}, although the one-shot intervention leaves the text rows at 16 bits. Scales and zero points (about $32/G = 0.25$ bits per element for fp16 pairs) and the convolution cache are not counted.

4.5 Caveats of the interventions

Zeroed keys still take attention. A zero key gives the logit $q^\top 0/\sqrt{d} = 0$, not $-\infty$, so each zeroed visual position keeps a softmax weight $w_i = 1/\mathcal{Z}$, with $\mathcal{Z}$ the softmax normaliser of the query, and contributes the zero value. The attention output becomes $\sum_{i \notin \mathcal{V}} w_i v_i$ with $\sum_{i \notin \mathcal{V}} w_i = 1 - n_v/\mathcal{Z} < 1$: KV zero removes the visual content and also shrinks the attention output by the mass the zeroed rows absorb. Masking the visual positions would remove the content and renormalise,

Table 2: Carriers of image information after each intervention. ✓ intact, ∼ degraded, – removed. Conv: the last three visual tokens through the convolution cache.

Intervention	visual KV	state	conv
full	✓	✓	✓
KV Q_4, N_4	∼	✓	✓
KV 0	– (dilution)	✓	✓
state Q_b, Q_b^{EF}, N_8	✓	∼	✓
state 0	✓	–	✓
KV drop	–	✓	✓
state 0_C	✓	–	–
KV 0 + state 0	– (dilution)	–	✓
KV drop + state 0_C	–	–	–
deletion, keep κ	κn_v rows	κn_v tokens	✓

which is what the drop level does; [11] avoid zero keys for this reason. The difference between KV zero and KV drop measures the dilution.

The convolution cache leaks the last visual tokens. Unlike [11], who drop the convolution cache together with the state, we leave it in place, following [12]. C_e^ℓ holds the inputs of the last positions of the span, and the causal convolution of kernel $\omega = 4$ mixes them into the GDN inputs of the first $\omega - 1 = 3$ tokens after the span. State zero therefore leaves a short local channel from the last three visual tokens into the first question-side tokens (and, through their hidden states, into everything above them). The 0_C level removes this channel, and its difference from state zero measures it.

What each intervention leaves. Table 2 lists the carriers of image information left after each intervention. The prefix rows of the KV cache precede the image and carry none of it. Zeroing both memories leaves only the convolution channel and the attention dilution of the zeroed keys, so $\mathcal{R}$ of that cell bounds what these two side effects contribute. KV drop with state 0_C leaves no carrier at all; its $\mathcal{R}$ should be close to 0, and it serves as a check of the protocol (it differs from the blind anchor only in that the prompt keeps the vision start and end tokens and the original positions).

State zero also forgets the prefix. S_e^ℓ summarises P as well as $\mathcal{V}$; zeroing it removes both. The prefix is the fixed chat-template header, the same for every sample.

4.6 Predictions

We state what prior work predicts before reporting results. (P1) If exact retrieval goes through attention, as in text [11], then reading tasks (DocVQA, TextVQA, and the values read off charts in ChartQA) should lose most of the image's contribution under KV zero and little under state zero. (P2) A coarse judgement such as object presence (POPE) may survive through the state alone, as the gist of a text context does [11]. (P3) In the one-shot cells a single rounding error is injected after the image and then carried, and damped by the decay, through the unquantized updates of the question tokens; the repeated injection that made the state fragile under every-write quantization [12] does not occur, so their loss should be well below that of the every-write cells at the same bit-widths. (P4) Error feedback can only matter for images with $n_v > 256$ ($C > 1$). Section 6.6 reports the outcome of each.

5 Experimental Setup

Models. Qwen3.5-0.8B in all phases run (1, 2, 3 and 5), bf16 weights, the post-trained (non-base) checkpoint, and Qwen3.5-4B for a check of five conditions (Section 6.4). The planned 9B phase (phase 4) is not run in this version. On the MPS backend neither the fla kernels nor the fused causal convolution are available, so GDN and its convolution run in the library's PyTorch reference implementation; transformers 5.22.2 and torch 2.15.0, as recorded in every result row. The 4B check ran with transformers 5.22.1 and torch 2.14.1 (CUDA 13.0); its rows do not record which GDN kernel was used, but the hook counts confirm that every cache write went through the hooked methods.

Data. Fixed random subsets (seed 0) of DocVQA validation (200) [36], ChartQA test (200) [34], TextVQA validation (200) [37] and POPE (300) [38], from the lmms-lab copies used by lmms-eval [39] at pinned revisions; the sample ids are released. Every question gets the same instruction, "Answer the question using a single word or phrase.", in the chat template without a system prompt, rendered with `enable_thinking=False` so that the prompt closes an empty thinking block before the answer (non-thinking mode); the code rejects any prompt that ends in an open thinking block, since the 4B and 9B templates open one by default.

Conditions. Phase 1 (which memory): $f \in \{Q_4, N_4, 0\}$ with the state unaltered; the state levels Q_8, Q_8^{EF}, Q_6, Q_6^{EF}, N_8 and 0 with the KV unaltered; and both zeroed. Phase 2 (bits): the state levels alone and with $f = Q_4$, and the two every-write cells (KV4 with Q_8 and Q_8^{EF}). Phase 3 (tokens versus bits): $\kappa \in \{1, 0.5, 0.25\}$ uniform, with KV at 16 or 4 bits, and $\kappa = 0.5$ random. Phase 5 (clean removal): KV drop, state 0_C, and both. (Phase 4, five conditions on 9B, is not run.) Each condition also has the two anchors per task. Batch size 1 and greedy decoding throughout.

Compute. One Apple M3 Pro laptop with 18 GB of unified memory (MPS backend) for 0.8B: the four phases evaluated 23,400 samples in 4.7 hours of summed per-sample time, 0.54–0.81 s per sample by phase. The 4B check took 2,800 samples at 0.58 s each on one NVIDIA A100-SXM4-40GB (Google Colab, about 4 compute units, peak 9.2 GB of GPU memory; Appendix C).

6 Results

All numbers are from Qwen3.5-0.8B on the full subsets (Section 5), except Section 6.4 (4B). The unaltered and blind accuracies are 0.880/0.053 (DocVQA), 0.745/0.100 (ChartQA), 0.726/0.062 (TextVQA) and 0.867/0.463 (POPE), so $\mathcal{R}$ is defined on every task; POPE has the smallest gap (0.40). Figure 2 shows both ratios for every condition and Table 3 the accuracy ratio; the loss ratio and the raw scores are in Appendix E.

6.1 Which memory carries the image

The KV rows are necessary for reading. Zeroing the visual KV rows leaves $\mathcal{R} = 0.02$, 0.02 and 0.07 on DocVQA, ChartQA and TextVQA, all with intervals that include or touch 0: the state, written by every visual token and larger than the visual KV rows of every image here, cannot supply the answer to a reading question on its own. On POPE it supplies about half ($\mathcal{R} = 0.53$ [0.37, 0.66]). Dropping the rows instead of zeroing them gives the same values (0.02, 0.03, 0.07, 0.54), so the softmax mass that zeroed keys absorb (Section 4.5) does not matter here, and KV zero can be read as removal.

Without the state, most of the gain remains, but less once the convolution cache goes too. Zeroing the state leaves $\mathcal{R} = 0.97$, 0.66, 0.93 and 0.91. That number includes the convolution channel: clearing the convolution cache together with the state (0_C) lowers it to 0.69, 0.33, 0.69 and 0.48 (Figure 3). The KV rows alone therefore carry well under the full gain; the state and the convolution cache contribute on every task, most on ChartQA. The size of the convolution effect is out of proportion to its width of three visual tokens. These interventions cannot tell whether those tokens carry image content into the question or whether clearing them mainly disturbs the GDN layers at the start of the question; the POPE cell with both memories zeroed, which keeps only this channel, retains 0.30 [0.13, 0.44], against ≤ 0 on the reading tasks.

The protocol closes. With the KV rows dropped and the state and convolution cache cleared, nothing that the image wrote is left, and $\mathcal{R}$ is -0.01, -0.01, -0.01 and -0.02, with every interval covering 0. Everything the image contributes reaches the answer through these three carriers.

Matched noise. Replacing the KV rows by noise of the 4-bit error's RMS is as harmless as 4-bit quantization ($\mathcal{R} \geq 0.98$ for both). State noise at the 8-bit error's RMS keeps 0.96–0.99, slightly below Q_8 (0.99–1.00), but within overlapping intervals; at these error sizes the structure of the rounding error does not matter.

6.2 How many bits each memory needs after reading

Quantizing the state once after the image costs nothing measurable at 8 or 6 bits ($\mathcal{R} = 0.98$–1.00), alone or together with 4-bit KV (0.96–1.01). With 4-bit KV and a 6-bit state the charged cache falls from 34.3 to 7.4 MB on DocVQA, 24.6 to 5.0 (ChartQA), 28.4 to 5.9 (TextVQA) and 22.6 to 4.5 MB (POPE) by (14). Error feedback during the image prefill makes no difference at either bit-width: every Q_b/Q_b^{EF} pair is within 0.03 and inside each other's interval, although almost every image spans several 256-token chunks (DocVQA: 188 of 200 images have five; median n_v is 1,240 for DocVQA, 425 for ChartQA, 768 for TextVQA and 270 for POPE). Quantizing every write of the whole run (KV4 with Q_8) instead of once costs 0.02 in $\mathcal{R}_{\mathrm{NLL}}$ on each of DocVQA, ChartQA and TextVQA (0.97–0.98 against 0.99–1.00; the intervals at most touch), and 0.01–0.03 in $\mathcal{R}$, inside the intervals.

6.3 Fewer visual tokens or fewer bits

The image's KV rows cost $\kappa\, b_{kv}$ bits per element and original visual token: 16, 8, 4 at $\kappa = 1, 0.5, 0.25$ with 16-bit KV, and 4, 2, 1 with 4-bit KV. The equal-memory pair is therefore $\kappa = 0.25$ at 16 bits against $\kappa = 1$ at 4 bits (4 bits each); the state (32 bits) is the same in both, and the text rows differ (16 versus 4 bits), which (14) charges. Figure 4 shows the whole grid. At 4 bits per element per visual token, quantizing keeps $\mathcal{R} = 1.00$, 1.00, 0.99 and 0.98, while deleting three quarters of the tokens keeps 0.31, 0.35 and 0.48 on the reading tasks and 0.90 on POPE. On the reading tasks, deletion loses 0.25–0.4 of the gain for every halving of the tokens, at either bit-width (Appendix E, Table 7), in line with the case made for visual KV quantization in transformer VLMs [18]. Our deletion is uniform or random before the image is read; attention-guided pruning [1] is not compared here. Uniform and random deletion at $\kappa = 0.5$ agree within their intervals (0.69 against 0.62 on DocVQA, 0.73 against 0.78 on TextVQA). Object presence is the exception: POPE keeps 0.88–0.90 at a quarter of the tokens, consistent with POPE keeping half of its gain with no visual KV rows at all.

6.4 Check at 4B

We repeated five conditions on Qwen3.5-4B on the first 100 samples of each subset, on one A100 (Colab; Table 4). The KV rows are again necessary for reading: KV zero leaves $\mathcal{R} = 0.05$, 0.06 and 0.06, with intervals that include 0, as at 0.8B (0.02–0.07). POPE keeps 0.41 [0.21, 0.67] without them; its unaltered–blind gap is only 0.29 at 4B, hence the wide interval. The state is less needed than at 0.8B: zeroing it, with the convolution cache left in place, keeps $\mathcal{R} = 1.00$–1.07 on every task, including ChartQA, where 0.8B kept only 0.66, and the loss ratio agrees on the reading tasks (1.00–1.01). 4-bit KV, the 8-bit state and both together

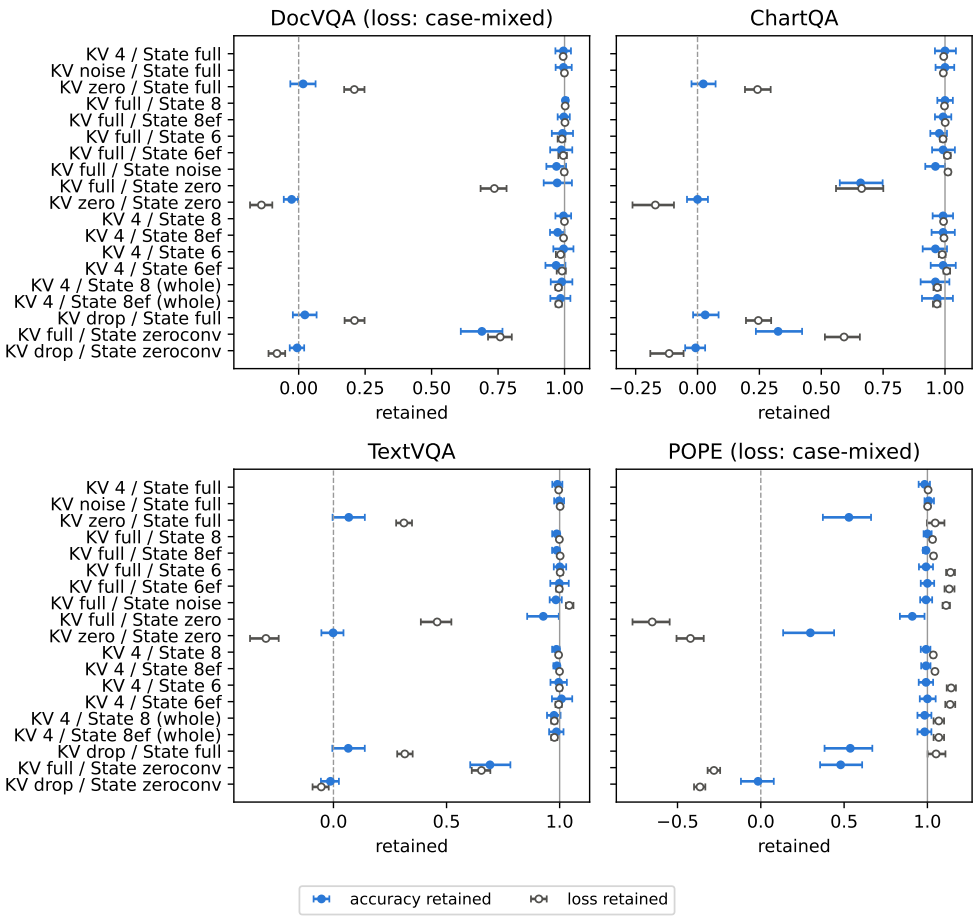


Figure 2: Accuracy retained $\mathcal{R}$ (filled) and loss retained $\mathcal{R}_{\mathrm{NLL}}$ (open) with 95% paired bootstrap intervals, Qwen3.5-0.8B. 1: the altered cache supplies the image's full gain; 0 (dashed): none of it. The loss ratio on DocVQA and POPE mixes in the letter case of the reference (Section 4.3).

are lossless ($\mathcal{R} \geq 0.99$). The form effect of Section 6.5 recurs: the unaltered 4B model answers POPE in lower case throughout, and after state zero 68% of its answers are capitalised ($\mathcal{R}_{\mathrm{NLL}} = 0.58$ at $\mathcal{R} = 1.07$). With 100 samples per task these intervals are wider than at 0.8B, and the state–convolution split (phase 5) was not run at 4B.

6.5 Task scores and the loss ratio

On the three reading tasks accuracy and the loss of the reference order the conditions alike, but not on one scale: $\mathcal{R}_{\mathrm{NLL}}$ is higher than $\mathcal{R}$ when the visual KV rows are removed (0.21–0.32 against 0.02–0.07) and lower when the state is (Figure 5, Appendix D). On POPE they disagree in sign: state zero keeps $\mathcal{R} = 0.91$ but has $\mathcal{R}_{\mathrm{NLL}} = -0.65$, and the 6-bit state has $\mathcal{R}_{\mathrm{NLL}} = 1.14$ at $\mathcal{R} = 0.99$. The greedy answers explain it. The reference is the lower-case word *yes* or *no*; the unaltered model writes it capitalised in 40% of samples, the blind model in 100%, and after state zero in 91%, after Q_6 in 26%. The loss of the lower-case reference tracks this shift in form, not the correctness of the answer. That the state controls the form of the answer parallels the text-only finding that the output language and a persona survive through the recurrent state [11]. POPE F1 falls further than accuracy when the visual KV rows are removed (0.55 against 0.68 under KV zero) and orders the removal conditions somewhat differently (Appendix E, Table 6).

6.6 The predictions

(P1) Holds: reading tasks lose almost all of the gain without the visual KV rows (0.02–0.07) and keep most of it without the state (0.66–0.97). (P2) Holds in part: POPE keeps half of its gain from the state alone (0.53), far more than reading tasks, but not all of it. (P3) Holds, with a small margin at these bit-widths: every-write quantization costs 0.02 of $\mathcal{R}_{\mathrm{NLL}}$ more than the one-shot cell, with intervals that at most touch, and nothing measurable in accuracy. (P4) Not supported in its premise: error feedback has nothing to correct, since one-shot Q_8 and Q_6 are already lossless, although nearly all images

Table 3: Accuracy retained $\mathcal{R}$ [95% CI] per condition and task, Qwen3.5-0.8B. KV and state levels as in Section 4.2; "ef": error feedback; "whole": every-write quantization of the whole run.

condition	DocVQA	ChartQA	TextVQA	POPE
KV 4 / State full	1.00 [0.97, 1.02]	1.00 [0.96, 1.04]	0.99 [0.97, 1.01]	0.98 [0.95, 1.02]
KV noise / State full	1.00 [0.97, 1.03]	1.00 [0.96, 1.04]	1.00 [0.98, 1.02]	1.01 [0.98, 1.04]
KV zero / State full	0.02 [-0.03, 0.06]	0.02 [-0.02, 0.07]	0.07 [-0.00, 0.14]	0.53 [0.37, 0.66]
KV full / State 8	1.00 [1.00, 1.01]	1.00 [0.97, 1.03]	0.99 [0.97, 1.00]	1.00 [0.98, 1.03]
KV full / State 8ef	1.00 [0.97, 1.02]	0.99 [0.96, 1.03]	0.99 [0.97, 1.00]	0.99 [0.97, 1.00]
KV full / State 6	0.99 [0.95, 1.03]	0.98 [0.94, 1.01]	1.00 [0.97, 1.03]	0.99 [0.95, 1.03]
KV full / State 6ef	0.99 [0.95, 1.03]	0.99 [0.95, 1.04]	1.00 [0.96, 1.04]	1.00 [0.96, 1.04]
KV full / State noise	0.97 [0.93, 1.01]	0.96 [0.92, 1.00]	0.98 [0.96, 1.01]	0.99 [0.96, 1.03]
KV full / State zero	0.97 [0.92, 1.03]	0.66 [0.57, 0.75]	0.93 [0.86, 1.00]	0.91 [0.83, 0.98]
KV zero / State zero	-0.03 [-0.06, -0.00]	0.00 [-0.04, 0.04]	-0.00 [-0.05, 0.04]	0.30 [0.13, 0.44]
KV 4 / State 8	1.00 [0.97, 1.02]	0.99 [0.95, 1.03]	0.99 [0.97, 1.00]	0.99 [0.96, 1.02]
KV 4 / State 8ef	0.97 [0.95, 1.00]	0.99 [0.95, 1.04]	0.99 [0.97, 1.00]	0.99 [0.96, 1.02]
KV 4 / State 6	1.00 [0.96, 1.03]	0.96 [0.91, 1.01]	1.00 [0.96, 1.03]	0.99 [0.95, 1.03]
KV 4 / State 6ef	0.97 [0.93, 1.00]	0.99 [0.94, 1.04]	1.01 [0.97, 1.06]	1.00 [0.95, 1.05]
KV 4 / State 8 (whole)	0.99 [0.95, 1.03]	0.96 [0.90, 1.02]	0.98 [0.94, 1.00]	0.98 [0.94, 1.02]
KV 4 / State 8ef (whole)	0.98 [0.95, 1.02]	0.97 [0.91, 1.03]	0.99 [0.95, 1.02]	0.98 [0.94, 1.02]
KV drop / State full	0.02 [-0.02, 0.07]	0.03 [-0.02, 0.09]	0.07 [-0.00, 0.14]	0.54 [0.38, 0.67]
KV full / State zeroconv	0.69 [0.61, 0.77]	0.33 [0.24, 0.42]	0.69 [0.60, 0.78]	0.48 [0.36, 0.61]
KV drop / State zeroconv	-0.01 [-0.03, 0.02]	-0.01 [-0.05, 0.03]	-0.01 [-0.06, 0.02]	-0.02 [-0.12, 0.08]

Table 4: Accuracy retained $\mathcal{R}$ [95% CI] on Qwen3.5-4B (first 100 samples per task; A100), with the 0.8B value on the full subsets in parentheses. "State zero" leaves the convolution cache.

condition	DocVQA	ChartQA	TextVQA	POPE
unaltered / blind accuracy	0.92 / 0.06	0.80 / 0.11	0.79 / 0.06	0.88 / 0.59
KV Q_4	1.00 [1.00, 1.00] (1.00)	1.00 [1.00, 1.00] (1.00)	0.99 [0.97, 1.00] (0.99)	1.00 [1.00, 1.00] (0.98)
KV zero	0.05 [-0.01, 0.11] (0.02)	0.06 [-0.04, 0.14] (0.02)	0.06 [-0.02, 0.14] (0.07)	0.41 [0.21, 0.67] (0.53)
state Q_8	1.00 [1.00, 1.00] (1.00)	1.00 [0.96, 1.04] (1.00)	1.00 [1.00, 1.00] (0.99)	1.00 [1.00, 1.00] (1.00)
state zero	1.04 [1.00, 1.08] (0.97)	1.00 [0.94, 1.06] (0.66)	1.02 [0.97, 1.08] (0.93)	1.07 [0.95, 1.27] (0.91)
KV Q_4 + state Q_8^{EF}	1.00 [1.00, 1.00] (0.97)	1.00 [1.00, 1.00] (0.99)	1.01 [0.99, 1.03] (0.99)	1.00 [1.00, 1.00] (0.99)

span several chunks.

7 Discussion and Limitations

For serving hybrid VLMs, the results point one way. The state cannot stand in for visual KV rows on reading tasks, so visual-token pruning that relies on the recurrent state to keep what was deleted [13] should be expected to work for coarse judgements such as object presence and not for reading text. Quantizing the cache after the image is read is nearly free: 4-bit KV and a 6-bit state keep the full gain at a fifth of the cache, whereas spending the same KV budget on fewer tokens loses half or more of it. The state is the larger memory for every image here, so its precision matters for the budget, and 6 bits after reading are enough at this scale. For evaluation, a teacher-forced loss on a fixed reference string measures the form of the answer together with its content; an intervention on the state, which shapes the form, can then look harmful or helpful while the answers do not change.

Limitations. (i) KV zero keeps softmax mass on the zeroed rows and state zero leaves the convolution cache (Section 4.5); the drop and 0_C levels remove these side effects. (ii) Each intervention acts once, after the image; it does not model a cache that stays quantized while the answer is generated, except for the two every-write cells. (iii) $\mathcal{R}_{\mathrm{NLL}}$ uses the loss of one reference answer per sample, verbatim and with its letter case, so a reference that the model would phrase or case differently inflates both anchors (Section 4.3); $\mathcal{R}$ avoids this but is coarser, since each sample's score is close to binary. (iv) Removing a pathway can show what the model is able to fall back on rather than what it normally uses [22]: $\mathcal{R}$ under KV zero or state zero measures the information the other memory can supply on its own, not how much of it the intact model draws from each. (v) Subsets of 200–300 samples per task, one model for the full study (0.8B) and a 100-sample check of six conditions at 4B (Section 6.4), one image resolution cap. (viii) The removal experiments do not separate image content in the convolution cache from the disturbance

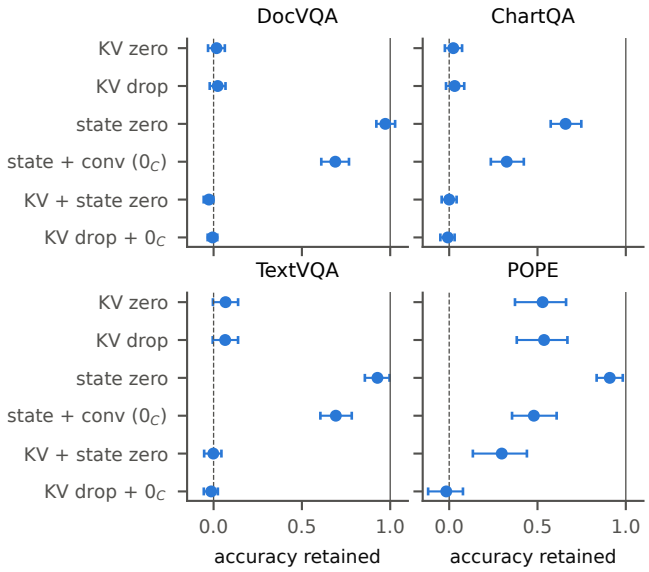


Figure 3: Accuracy retained when carriers of the image are removed (95% CI). "Zero" keeps the zeroed KV rows in the softmax; "drop" removes them. "State zero" leaves the convolution cache, which still holds the last three visual tokens; 0_C clears it too.

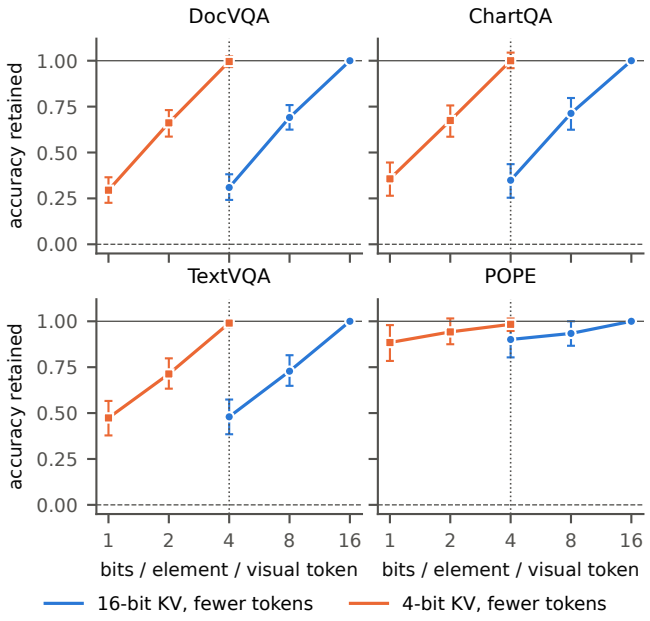


Figure 4: Accuracy retained against the bits that the image's KV rows cost per element and original visual token ($\kappa\, b_{kv}$), with 95% CI. Each line deletes visual tokens uniformly ($\kappa = 1, 0.5, 0.25$) at a fixed KV bit-width. Dotted: the equal-cost pair (all tokens at 4 bits; a quarter of the tokens at 16 bits).

of clearing it (Section 6.1). (vi) Quantization is simulated; no memory or speed is saved in these runs. (vii) All runs use the PyTorch reference kernels on MPS; bf16 numerics may differ slightly from the fused CUDA kernels used in deployment. The full set of conditions on 4B and 9B, whose memories are $8/3$ times larger with the same crossover, is left to future work.

8 Conclusion

In a hybrid VLM that reads an image into a softmax-attention KV cache and a Gated DeltaNet state at once, the KV rows of the visual tokens are what reading questions depend on: without them the state supplies almost nothing, while without the state most of the image's contribution survives, more so when the convolution cache passes on the last visual tokens. Object presence is shared more evenly. After reading, both memories tolerate low precision, and quantizing every visual token is far better than keeping fewer of them at the same cost. Interventions on the state change the form of answers as well as their content, which a loss-based measure confuses with information loss. At 4B the visual KV rows remain necessary for reading and the state becomes dispensable, so the picture sharpens rather than changes with scale.

Acknowledgments

This work was carried out by a single author without institutional funding; the 0.8B experiments ran on the author's laptop and the 4B check on self-funded Google Colab compute. The author used Claude (Anthropic) as an assistant for code development and manuscript editing; all experiments, analyses and claims were designed, run and verified by the author.

Reproducibility and Data Availability

Code, configurations, subset ids, raw per-sample results and the scripts that regenerate every table and figure are at `https://github.com/skytinstone/twomemories`. Each phase is run by `python scripts/run_phase.py --phase <name>`, the tables by `python analysis/make_tables.py` and the figures by `python analysis/figures.py`. Quantization uses the released `hybridquant` package [12] unchanged. All datasets are public.

References

[1] Liang Chen, Haozhe Zhao, Tianyu Liu, Shuai Bai, Junyang Lin, Chang Zhou, and Baobao Chang. An image is worth 1/2 tokens after layer 2: Plug-and-play inference acceleration for large vision-language models. In *European Conference on Computer Vision (ECCV)*, 2024. arXiv:2403.06764.

[2] Yuan Zhang, Chun-Kai Fan, Junpeng Ma, Wenzhao Zheng, Tao Huang, Kuan Cheng, Denis Gudovskiy,

Tomoyuki Okuno, Yohei Nakata, Kurt Keutzer, and Shanghang Zhang. SparseVLM: Visual token sparsification for efficient vision-language model inference. In *International Conference on Machine Learning (ICML)*, 2025. arXiv:2410.04417.

[3] Yuzhang Shang, Mu Cai, Bingxin Xu, Yong Jae Lee, and Yan Yan. LLaVA-PruMerge: Adaptive token reduction for efficient large multimodal models. In *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2025. arXiv:2403.15388.

[4] Senqiao Yang, Yukang Chen, Zhuotao Tian, Chengyao Wang, Jingyao Li, Bei Yu, and Jiaya Jia. VisionZip: Longer is better but not necessary in vision language models. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 19792–19802, 2025. arXiv:2412.04467.

[5] Long Xing, Qidong Huang, Xiaoyi Dong, Jiajie Lu, Pan Zhang, Yuhang Zang, Yuhang Cao, Conghui He, Jiaqi Wang, Feng Wu, and Dahua Lin. PyramidDrop: Accelerating your large vision-language models via pyramid visual redundancy reduction. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025. arXiv:2410.17247.

[6] Zhongwei Wan, Ziang Wu, Che Liu, Jinfa Huang, Zhihong Zhu, Peng Jin, Longyue Wang, and Li Yuan. LOOK-M: Look-once optimization in KV cache for efficient multimodal long-context inference. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, pages 4065–4078, 2024. arXiv:2406.18139.

[7] Dezhan Tu, Danylo Vashchilenko, Yuzhe Lu, and Panpan Xu. VL-Cache: Sparsity and modality-aware KV cache compression for vision-language model inference acceleration. In *International Conference on Learning Representations (ICLR)*, 2025. arXiv:2410.23317.

[8] Qwen Team. Qwen3.5: Towards native multimodal agents. `https://qwen.ai/blog?id=qwen3.5`, February 2026.

[9] Songlin Yang, Jan Kautz, and Ali Hatamizadeh. Gated delta networks: Improving Mamba2 with delta rule. In *International Conference on Learning Representations (ICLR)*, 2025. arXiv:2412.06464.

[10] vLLM contributors. [rfc]: fp8 kv cache gives little to no memory benefit on mamba/gdn hybrid models. `https://github.com/vllm-project/vllm/issues/55196`, 2026.

[11] Kirill Afendulev, Alexey Dontsov, Elena Tutubalina, and Anton Korznikov. What attention recalls and recurrence controls in hybrid language models. In *Findings of the Association for Computational Linguistics: EMNLP 2026*, 2026. arXiv:2609.04434.

[12] Minseok Shin. Joint KV-cache and recurrent-state quantization in hybrid linear-attention LLMs, 2026. Unpublished manuscript; code at `https://github.com/skytinstone/hybridquant`.

[13] Jindong Jiang, Amala Sanjay Deshmukh, Kateryna Chumachenko, Karan Sapra, Zhiding Yu, Guilin Liu, Andrew Tao, Pavlo Molchanov, Jan Kautz, and Wonmin Byeon. Stateful token reduction for long-video hybrid VLMs. *arXiv preprint arXiv:2603.00198*, 2026.

[14] Daniel Bolya, Cheng-Yang Fu, Xiaoliang Dai, Peizhao Zhang, Christoph Feichtenhofer, and Judy Hoffman. Token merging: Your ViT but faster. In *International Conference on Learning Representations (ICLR)*, 2023. arXiv:2210.09461.

[15] Seong Yeol Park, Min Jung Kwon, Xianghua Piao, and Yeong Hyeon Gu. DTP: Delta-guided two stage pruning for Mamba-based multimodal large language models. In *International Conference on Learning Representations (ICLR)*, 2026.

[16] Zirui Liu, Jiayi Yuan, Hongye Jin, Shaochen Zhong, Zhaozhuo Xu, Vladimir Braverman, Beidi Chen, and Xia Hu. Kivi: A tuning-free asymmetric 2bit quantization for kv cache. In *ICML*, 2024. arXiv:2402.02750.

[17] Coleman Hooper, Sehoon Kim, Hiva Mohammadzadeh, Michael W. Mahoney, Yakun Sophia Shao, Kurt Keutzer, and Amir Gholami. Kvquant: Towards 10 million context length llm inference with kv cache quantization. In *NeurIPS*, 2024. arXiv:2401.18079.

[18] Insu Han, Zeliang Zhang, Zhiyuan Wang, Yifan Zhu, Susan Liang, Jiani Liu, Haiting Lin, Mingjie Zhao, Chenliang Xu, Kun Wan, and Wentian Zhao. CalibQuant: 1-bit KV cache quantization for multimodal LLMs. *arXiv preprint arXiv:2502.14882*, 2025.

[19] Omri Kaduri, Shai Bagon, and Tali Dekel. What's in the image? A deep-dive into the vision of vision language models. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025. arXiv:2411.17491.

[20] Zhi Zhang, Srishti Yadav, Fengze Han, and Ekaterina Shutova. Cross-modal information flow in multimodal large language models. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025. arXiv:2411.18620.

[21] Clement Neo, Luke Ong, Philip Torr, Mor Geva, David Krueger, and Fazl Barez. Towards interpreting visual information processing in vision-language models. In *International Conference on Learning Representations (ICLR)*, 2025. arXiv:2410.07149.

[22] Israfel Salazar, Stella Frank, Dan Oneata, Desmond Elliott, and Constanza Fierro. Pathways of visual information flow in vision-language models. *arXiv preprint arXiv:2607.03358*, 2026.

[23] Taeyoun Kwon, Seungjin Kim, Hyeonyu Kim, and Moon Hwan Kim. DeltaS: Reading the gated linear attention state for KV cache eviction in streaming video. *arXiv preprint arXiv:2609.27470*, 2026.

[24] Han Zhao, Min Zhang, Wei Zhao, Pengxiang Ding, Siteng Huang, and Donglin Wang. Cobra: Extending Mamba to multi-modal large language model for efficient inference. In *AAAI Conference on Artificial Intelligence (AAAI)*, 2025. arXiv:2403.14520.

[25] Yanyuan Qiao, Zheng Yu, Longteng Guo, Sihan Chen, Zijia Zhao, Mingzhen Sun, Qi Wu, and Jing Liu. VL-Mamba: Exploring state space models for multimodal learning. In *NeurIPS Efficient Natural Language and Speech Processing Workshop (ENLSP-IV), PMLR 262*, pages 102–113, 2024. arXiv:2403.13600.

[26] Hongyuan Tao, Bencheng Liao, Shaoyu Chen, Haoran Yin, Qian Zhang, Wenyu Liu, and Xinggang Wang. InfiniteVL: Synergizing linear and sparse attention for highly-efficient, unlimited-input vision-language models. *arXiv preprint arXiv:2512.08829*, 2025.

[27] Tao Zhang, Jianchao Tan, Pingwei Sun, Yanqi Yu, Zunhai Su, Zixu Jiang, Yuchen Xie, Xunliang Cai, and Ziqian Zeng. Damp: Decay-aware mixed-precision recurrent-state quantization. *arXiv preprint arXiv:2608.27513*, 2026.

[28] Bingchen Yao, Haobo Xu, Haokun Lin, Yichen Wu, Ziyu Guo, Renrui Zhang, Zhichao Lu, Zhenan Sun, and Ying Wei. Stepquant: When and where errors matter in delta-rule recurrent state quantization. *arXiv preprint arXiv:2609.38169*, 2026.

[29] Yi Pan, Haocheng Xi, Kan Zhu, Xingyang Li, Yibo Wu, Mayank Mishra, Hongtao Zhang, William X. Zheng, Baris Kasikci, Song Han, Kurt Keutzer, Rishabh Iyer, and Ion Stoica. Leapquant: Efficient linear attention with accurate recurrent state quantization. *arXiv preprint arXiv:2609.38166*, 2026.

[30] Peng Wang, Shuai Bai, Sinan Tan, Shijie Wang, Zhihao Fan, Jinze Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, Yang Fan, Kai Dang, Mengfei Du, Xuancheng Ren, Rui Men, Dayiheng Liu, Chang Zhou, Jingren Zhou, and Junyang Lin. Qwen2-VL: Enhancing vision-language model's perception of the world at any resolution. *arXiv preprint arXiv:2409.12191*, 2024.

[31] Bradley Efron and Robert J. Tibshirani. *An Introduction to the Bootstrap*, volume 57 of *Monographs on Statistics and Applied Probability*. Chapman & Hall/CRC, 1993.

[32] Ali Furkan Biten, Ruben Tito, Andres Mafla, Lluis Gomez, Marçal Rusiñol, Ernest Valveny, C. V. Jawahar, and Dimosthenis Karatzas. Scene text visual question answering. In *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2019. arXiv:1905.13648.

[33] Nitesh Methani, Pritha Ganguly, Mitesh M. Khapra, and Pratyush Kumar. PlotQA: Reasoning over scientific plots. In *IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, 2020. arXiv:1909.00997.

[34] Ahmed Masry, Do Xuan Long, Jia Qing Tan, Shafiq Joty, and Enamul Hoque. ChartQA: A benchmark for question answering about charts with visual and logical reasoning. In *Findings of the Association for Computational Linguistics: ACL 2022*, 2022. arXiv:2203.10244.

[35] Stanislaw Antol, Aishwarya Agrawal, Jiasen Lu, Margaret Mitchell, Dhruv Batra, C. Lawrence Zitnick, and Devi Parikh. VQA: Visual question answering. In *IEEE International Conference on Computer Vision (ICCV)*, pages 2425–2433, 2015. arXiv:1505.00468.

[36] Minesh Mathew, Dimosthenis Karatzas, and C. V. Jawahar. DocVQA: A dataset for VQA on document images. In *IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, 2021. arXiv:2007.00398.

[37] Amanpreet Singh, Vivek Natarajan, Meet Shah, Yu Jiang, Xinlei Chen, Dhruv Batra, Devi Parikh, and Marcus Rohrbach. Towards VQA models that can read. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019. arXiv:1904.08920.

[38] Yifan Li, Yifan Du, Kun Zhou, Jinpeng Wang, Wayne Xin Zhao, and Ji-Rong Wen. Evaluating object hallucination in large vision-language models. In *Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023. arXiv:2305.10355.

[39] Kaichen Zhang, Bo Li, Peiyuan Zhang, Fanyi Pu, Joshua Adrian Cahyono, Kairui Hu, Shuai Liu, Yuanhan Zhang, Jingkang Yang, Chunyuan Li, and Ziwei Liu. LMMs-Eval: Reality check on the evaluation of large multimodal models. In *Findings of the Association for Computational Linguistics: NAACL 2025*, pages 881–916, 2025. arXiv:2407.12772.

A Implementation premises

Checked against transformers 5.22.2 (Qwen3.5 modeling and cache code) before any run; file and line references are in `docs/assumptions.md` of the repository.

	Premise	Consequence for the method
1	Visual features are scattered into the input embedding only; no intermediate-layer injection.	The caches after the span (KV, state, convolution cache) are the only carriers of the image.
2	The image is one contiguous span of placeholder tokens of length $\prod(\text{grid})/m^2$.	$\mathcal{V}$ is known exactly; (1).
3	The recurrent state is stored in fp32 (PyTorch path).	State charged at 32 bits; (5). The PyTorch path is the one used in all runs.
4	The cache is a `DynamicCache` with attention and linear-attention layers; the write methods hooked by `hybridquant` exist unchanged.	No adapter; every-write cells use the hooks as released.
5	Prefill computes GDN in 64-token chunks internally but writes the state once per forward call.	Error feedback during the image needs one forward call per 256-token chunk (9).

B Conditions

The 22 conditions of phases 1–3 and the three of phase 5, run on Qwen3.5-0.8B, with the phases that run them; the five marked P4 form the planned 9B phase, which is not run in this version. Bits are the charges of (14). Every task also has the blind anchor.

C Compute

0.8B: one Apple M3 Pro laptop (18 GB unified memory, MPS backend); 4B: one NVIDIA A100-SXM4-40GB on Google Colab (4.0 compute units by the logged estimate). Seconds are summed per-sample wall time of the evaluation (two-stage prefill, teacher-forced pass, greedy decoding) from `results/{mac,colab}/compute_log.jsonl`, excluding model loading and data download. Cells shared between phases are run once and counted in the first phase that runs them. The process footprint, including MPS memory, stayed near 8 GB as read by the live monitor; it is not recorded in the result rows.

Run	Model	samples	s/sample	hours
phase 1	0.8B	10,800	0.73	2.20
phase 2	0.8B	5,400	0.81	1.21
phase 3	0.8B	4,500	0.54	0.67
phase 5	0.8B	2,700	0.78	0.59
total	0.8B	23,400	0.72	4.67
check (4B, A100)	4B	2,800	0.58	0.45

D Loss and accuracy on POPE

Figure 5 plots $\mathcal{R}_{\text{NLL}}$ against $\mathcal{R}$ for every condition and task. Points on the diagonal agree. ChartQA lies on it; DocVQA and TextVQA leave it under state zero, where the loss ratio is lower (0.74 and 0.46 at $\mathcal{R} = 0.97$ and 0.93); POPE leaves it in both directions. On POPE the share of greedy answers that start with a capital letter is 0.40 unaltered, 1.00 blind, 0.91 after state zero, 0.96 after state 0_C, 0.36 after KV zero and 0.26 after Q_6, while the reference is always lower case.

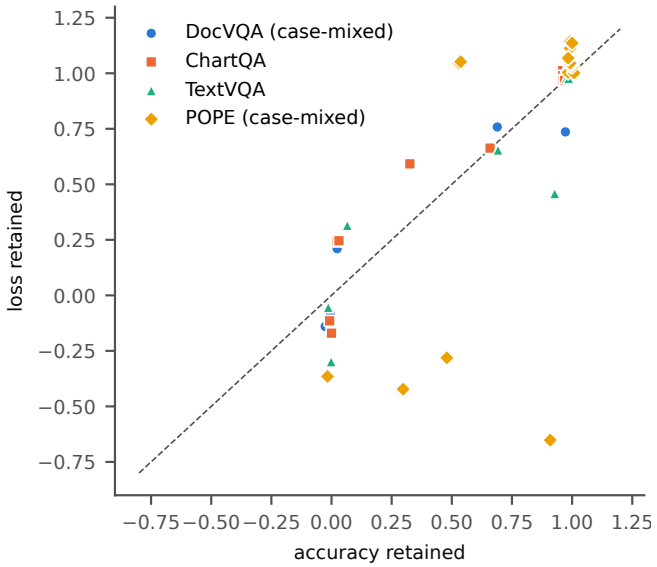


Figure 5: Loss retained against accuracy retained, one point per condition and task (Qwen3.5-0.8B). Dashed: equality.

E Full results

Generated by `analysis/make_tables.py` from `results/mac`. †: the loss ratio mixes in the letter case of the reference (Section 4.3).

Label	KV f	state g	tokens	scope	phases	b_{kv}	b_s
KVfull_Sfull	id	id	all	image	P1–P4	16	32
KV4_Sfull	Q_4	id	all	image	P1, P2, P3	4	32
KVnoise_Sfull	N_4	id	all	image	P1	4	32
KVzero_Sfull	0	id	all	image	P1, P4	0	32
KVfull_S8	id	Q_8	all	image	P1, P2	16	8
KVfull_S8ef	id	Q_8^{EF}	all	image	P1, P2	16	8
KVfull_S6	id	Q_6	all	image	P1, P2	16	6
KVfull_S6ef	id	Q_6^{EF}	all	image	P1, P2	16	6
KVfull_Snoise	id	N_8	all	image	P1	16	8
KVfull_Szero	id	0	all	image	P1, P4	16	0
KVzero_Szero	0	0	all	image	P1	0	0
KV4_S8	Q_4	Q_8	all	image	P2	4	8
KV4_S8ef	Q_4	Q_8^{EF}	all	image	P2, P4	4	8
KV4_S6	Q_4	Q_6	all	image	P2	4	6
KV4_S6ef	Q_4	Q_6^{EF}	all	image	P2	4	6
KV4_S8_whole	Q_4	Q_8	all	whole	P2	4	8
KV4_S8ef_whole	Q_4	Q_8^{EF}	all	whole	P2	4	8
KVfull_Sfull_T50u	id	id	50% uniform	image	P3, P4	16	32
KV4_Sfull_T50u	Q_4	id	50% uniform	image	P3	4	32
KVfull_Sfull_T25u	id	id	25% uniform	image	P3	16	32
KV4_Sfull_T25u	Q_4	id	25% uniform	image	P3	4	32
KVfull_Sfull_T50r	id	id	50% random	image	P3	16	32
KVdrop_Sfull	drop	id	all	image	P5	0	32
KVfull_Szeroconv	id	0_C	all	image	P5	16	0
KVdrop_Szeroconv	drop	0_C	all	image	P5	0	0

Table 5: Loss retained $\mathcal{R}_{\mathrm{NLL}}$ [95% CI], Qwen3.5-0.8B.

condition	DocVQA†	ChartQA	TextVQA	POPE†
KV 4 / State full	0.99 [0.99, 1.00]	0.99 [0.99, 1.00]	1.00 [0.99, 1.00]	1.00 [1.00, 1.01]
KV noise / State full	1.00 [0.99, 1.00]	0.99 [0.99, 1.00]	1.00 [1.00, 1.01]	1.00 [1.00, 1.01]
KV zero / State full	0.21 [0.17, 0.25]	0.24 [0.19, 0.30]	0.31 [0.28, 0.35]	1.05 [1.00, 1.10]
KV full / State 8	1.00 [1.00, 1.01]	1.00 [0.99, 1.00]	1.00 [0.99, 1.00]	1.03 [1.02, 1.04]
KV full / State 8ef	1.00 [1.00, 1.01]	1.00 [1.00, 1.01]	1.00 [1.00, 1.01]	1.04 [1.02, 1.05]
KV full / State 6	0.99 [0.97, 1.00]	0.99 [0.98, 1.00]	1.00 [0.99, 1.01]	1.14 [1.11, 1.17]
KV full / State 6ef	1.00 [0.98, 1.01]	1.01 [1.00, 1.02]	1.00 [0.99, 1.01]	1.13 [1.10, 1.16]
KV full / State noise	1.00 [0.99, 1.01]	1.01 [1.00, 1.02]	1.04 [1.03, 1.06]	1.11 [1.09, 1.14]
KV full / State zero	0.74 [0.68, 0.78]	0.66 [0.56, 0.75]	0.46 [0.39, 0.52]	-0.65 [-0.77, -0.55]
KV zero / State zero	-0.14 [-0.18, -0.10]	-0.17 [-0.26, -0.10]	-0.30 [-0.37, -0.24]	-0.42 [-0.51, -0.34]
KV 4 / State 8	1.00 [0.99, 1.01]	0.99 [0.99, 1.00]	1.00 [0.99, 1.00]	1.04 [1.02, 1.05]
KV 4 / State 8ef	1.00 [0.99, 1.00]	1.00 [0.99, 1.00]	1.00 [1.00, 1.00]	1.05 [1.03, 1.06]
KV 4 / State 6	0.99 [0.97, 1.00]	0.99 [0.98, 1.00]	1.00 [0.99, 1.01]	1.14 [1.12, 1.17]
KV 4 / State 6ef	0.99 [0.97, 1.00]	1.01 [0.99, 1.02]	1.00 [0.98, 1.01]	1.14 [1.11, 1.17]
KV 4 / State 8 (whole)	0.98 [0.97, 0.99]	0.97 [0.95, 0.98]	0.98 [0.96, 0.99]	1.07 [1.04, 1.10]
KV 4 / State 8ef (whole)	0.98 [0.97, 0.99]	0.97 [0.95, 0.98]	0.98 [0.96, 0.99]	1.07 [1.04, 1.10]
KV drop / State full	0.21 [0.17, 0.25]	0.25 [0.20, 0.30]	0.32 [0.28, 0.35]	1.05 [1.00, 1.11]
KV full / State zeroconv	0.76 [0.71, 0.80]	0.59 [0.51, 0.66]	0.65 [0.61, 0.69]	-0.28 [-0.32, -0.24]
KV drop / State zeroconv	-0.08 [-0.11, -0.05]	-0.11 [-0.19, -0.06]	-0.05 [-0.09, -0.02]	-0.37 [-0.40, -0.33]

Table 6: Task scores: ANLS (DocVQA), relaxed accuracy (ChartQA), VQA accuracy (TextVQA), accuracy / F1 (POPE).

condition	DocVQA	ChartQA	TextVQA	POPE
unaltered	0.880	0.745	0.726	0.867 / 0.856
blind (no image)	0.053	0.100	0.062	0.463 / 0.462
KV 4 / State full	0.877	0.745	0.720	0.860 / 0.850
KV noise / State full	0.877	0.745	0.725	0.870 / 0.860
KV zero / State full	0.066	0.115	0.107	0.677 / 0.553
KV full / State 8	0.883	0.745	0.718	0.867 / 0.856
KV full / State 8ef	0.878	0.740	0.718	0.863 / 0.852
KV full / State 6	0.874	0.730	0.727	0.863 / 0.854
KV full / State 6ef	0.870	0.740	0.725	0.867 / 0.857
KV full / State noise	0.855	0.720	0.715	0.863 / 0.851
KV full / State zero	0.857	0.525	0.678	0.830 / 0.805
KV zero / State zero	0.031	0.100	0.061	0.583 / 0.309
KV 4 / State 8	0.877	0.740	0.717	0.863 / 0.854
KV 4 / State 8ef	0.859	0.740	0.718	0.863 / 0.853
KV 4 / State 6	0.877	0.720	0.723	0.863 / 0.854
KV 4 / State 6ef	0.854	0.740	0.732	0.867 / 0.857
KV 4 / State 8 (whole)	0.872	0.720	0.710	0.860 / 0.846
KV 4 / State 8ef (whole)	0.868	0.725	0.717	0.860 / 0.846
KV drop / State full	0.072	0.120	0.106	0.680 / 0.560
KV full / State zeroconv	0.623	0.310	0.521	0.657 / 0.713
KV drop / State zeroconv	0.048	0.095	0.053	0.457 / 0.407

Table 7: Visual tokens kept (uniform) × KV bits: charged MB (14) and both retained ratios. MB group: totals within 10%; the 18.9 MB state dominates every total.

task	tokens	KV bits	total MB	acc retained [95% CI]	loss retained [95% CI]	MB group
DocVQA†	100%	full	34.25	1.00 [1.00, 1.00]	1.00 [1.00, 1.00]	A
DocVQA†	100%	4	22.72	1.00 [0.97, 1.02]	0.99 [0.99, 1.00]	B
DocVQA†	50%	full	26.78	0.69 [0.62, 0.76]	0.80 [0.76, 0.83]	C
DocVQA†	50%	4	20.85	0.66 [0.59, 0.73]	0.79 [0.75, 0.83]	B
DocVQA†	25%	full	23.05	0.31 [0.24, 0.38]	0.44 [0.38, 0.51]	B
DocVQA†	25%	4	19.92	0.29 [0.23, 0.37]	0.43 [0.37, 0.50]	D
ChartQA	100%	full	24.64	1.00 [1.00, 1.00]	1.00 [1.00, 1.00]	A
ChartQA	100%	4	20.32	1.00 [0.96, 1.04]	0.99 [0.99, 1.00]	B
ChartQA	50%	full	22.00	0.71 [0.62, 0.80]	0.74 [0.69, 0.79]	B
ChartQA	50%	4	19.66	0.67 [0.59, 0.76]	0.73 [0.67, 0.78]	B
ChartQA	25%	full	20.69	0.35 [0.25, 0.44]	0.41 [0.30, 0.49]	B
ChartQA	25%	4	19.33	0.36 [0.26, 0.45]	0.41 [0.30, 0.49]	B
TextVQA	100%	full	28.43	1.00 [1.00, 1.00]	1.00 [1.00, 1.00]	A
TextVQA	100%	4	21.26	0.99 [0.97, 1.01]	1.00 [0.99, 1.00]	B
TextVQA	50%	full	23.85	0.73 [0.65, 0.82]	0.75 [0.71, 0.79]	C
TextVQA	50%	4	20.12	0.71 [0.63, 0.80]	0.75 [0.70, 0.78]	B
TextVQA	25%	full	21.57	0.48 [0.38, 0.57]	0.51 [0.44, 0.57]	B
TextVQA	25%	4	19.55	0.47 [0.38, 0.57]	0.50 [0.43, 0.56]	B
POPE†	100%	full	22.58	1.00 [1.00, 1.00]	1.00 [1.00, 1.00]	A
POPE†	100%	4	19.80	0.98 [0.95, 1.02]	1.00 [1.00, 1.01]	B
POPE†	50%	full	20.93	0.93 [0.87, 1.00]	0.34 [0.28, 0.40]	A
POPE†	50%	4	19.39	0.94 [0.88, 1.02]	0.34 [0.28, 0.40]	B
POPE†	25%	full	20.11	0.90 [0.80, 1.00]	0.24 [0.18, 0.31]	B
POPE†	25%	4	19.18	0.88 [0.78, 0.98]	0.24 [0.18, 0.31]	B